

Taskbook

Full-Stack Todo Application — Source Code

Stack: React + Vite • Express 5 • PostgreSQL • Drizzle ORM

Generated: Wed May 27 2026

Table of Contents

1. Database Schema	lib/db/src/schema/todos.ts
2. OpenAPI Spec	lib/api-spec/openapi.yaml
3. API Routes — todos.ts	artifacts/api-server/src/routes/todos.ts
4. App Entry — App.tsx	artifacts/todo-app/src/App.tsx
5. Layout — layout.tsx	artifacts/todo-app/src/components/layout.tsx
6. Home Page — home.tsx	artifacts/todo-app/src/pages/home.tsx
7. Calendar Page — calendar.tsx	artifacts/todo-app/src/pages/calendar.tsx
8. Stats Page — stats.tsx	artifacts/todo-app/src/pages/stats.tsx
9. Create Todo — create-todo.tsx	artifacts/todo-app/src/components/todo/create-todo.tsx
10. Todo Item — todo-item.tsx	artifacts/todo-app/src/components/todo/todo-item.tsx
11. Todo List — todo-list.tsx	artifacts/todo-app/src/components/todo/todo-list.tsx

Database Schema

lib/db/src/schema/todos.ts

```
1  import { pgTable, text, serial, timestamp, boolean } from "drizzle-orm/pg-core";
2  import { createInsertSchema } from "drizzle-zod";
3  import { z } from "zod/v4";
4
5  export const todosTable = pgTable("todos", {
6    id: serial("id").primaryKey(),
7    title: text("title").notNull(),
8    description: text("description"),
9    completed: boolean("completed").notNull().default(false),
10   priority: text("priority", { enum: ["low", "medium", "high"] }).notNull().default("medium"),
11   dueDate: text("due_date"),
12   createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
13   updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow().
14   $onUpdate(() => new Date()),
15 });
16
17 export const insertTodoSchema = createInsertSchema(todosTable).omit({
18   id: true,
19   createdAt: true,
20   updatedAt: true,
21 });
22
23 export type InsertTodo = z.infer<typeof insertTodoSchema>;
24 export type Todo = typeof todosTable.$inferSelect;
```

```
1  openapi: 3.1.0
2  info:
3    # Do not change the title, if the title changes, the import paths will be broken
4    title: Api
5    version: 0.1.0
6    description: Todo App API specification
7  servers:
8    - url: /api
9      description: Base API path
10 tags:
11   - name: health
12     description: Health operations
13   - name: todos
14     description: Todo operations
15 paths:
16   /healthz:
17     get:
18       operationId: healthCheck
19       tags: [health]
20       summary: Health check
21       description: Returns server health status
22       responses:
23         "200":
24           description: Healthy
25           content:
26             application/json:
27               schema:
28                 $ref: "#/components/schemas/HealthStatus"
29
30   /todos:
31     get:
32       operationId: listTodos
33       tags: [todos]
34       summary: List all todos
35       parameters:
36         - name: status
37           in: query
38           required: false
39           schema:
40             type: string
41             enum: [all, active, completed]
42         - name: priority
43           in: query
44           required: false
45           schema:
46             type: string
47             enum: [low, medium, high]
48       responses:
49         "200":
50           description: List of todos
51           content:
52             application/json:
53               schema:
54                 type: array
55                 items:
56                   $ref: "#/components/schemas/Todo"
```

```
57     post:
58         operationId: createTodo
59         tags: [todos]
60         summary: Create a new todo
61         requestBody:
62             required: true
63             content:
64                 application/json:
65                     schema:
66                         $ref: "#/components/schemas/ToDoInput"
67         responses:
68             "201":
69                 description: Created todo
70                 content:
71                     application/json:
72                         schema:
73                             $ref: "#/components/schemas/ToDo"
74
75 /todos/clear-completed:
76     delete:
77         operationId: clearCompleted
78         tags: [todos]
79         summary: Delete all completed todos
80         responses:
81             "200":
82                 description: Number of deleted todos
83                 content:
84                     application/json:
85                         schema:
86                             $ref: "#/components/schemas/ClearCompletedResult"
87
88 /todos/stats:
89     get:
90         operationId: getToDoStats
91         tags: [todos]
92         summary: Get todo statistics
93         responses:
94             "200":
95                 description: Todo statistics summary
96                 content:
97                     application/json:
98                         schema:
99                             $ref: "#/components/schemas/ToDoStats"
100
101 /todos/{id}:
102     get:
103         operationId: getToDo
104         tags: [todos]
105         summary: Get a todo by ID
106         parameters:
107             - name: id
108               in: path
109               required: true
110               schema:
111                 type: integer
112         responses:
113             "200":
114                 description: Todo item
```

```
115         content:
116             application/json:
117                 schema:
118                     $ref: "#/components/schemas/ToDo"
119     "404":
120         description: Not found
121 patch:
122     operationId: updateTodo
123     tags: [todos]
124     summary: Update a todo
125     parameters:
126         - name: id
127           in: path
128           required: true
129           schema:
130               type: integer
131     requestBody:
132         required: true
133         content:
134             application/json:
135                 schema:
136                     $ref: "#/components/schemas/ToDoUpdate"
137     responses:
138         "200":
139             description: Updated todo
140             content:
141                 application/json:
142                     schema:
143                         $ref: "#/components/schemas/ToDo"
144         "404":
145             description: Not found
146 delete:
147     operationId: deleteTodo
148     tags: [todos]
149     summary: Delete a todo
150     parameters:
151         - name: id
152           in: path
153           required: true
154           schema:
155               type: integer
156     responses:
157         "204":
158             description: Deleted
159         "404":
160             description: Not found
161
162 /todos/{id}/complete:
163 patch:
164     operationId: toggleTodoComplete
165     tags: [todos]
166     summary: Toggle todo completion status
167     parameters:
168         - name: id
169           in: path
170           required: true
171           schema:
172               type: integer
```

```
173     responses:
174       "200":
175         description: Updated todo
176         content:
177           application/json:
178             schema:
179               $ref: "#/components/schemas/ToDo"
180       "404":
181         description: Not found
182
183   components:
184     schemas:
185       HealthStatus:
186         type: object
187         properties:
188           status:
189             type: string
190         required:
191           - status
192
193       ToDo:
194         type: object
195         required:
196           - id
197           - title
198           - completed
199           - priority
200           - createdAt
201         properties:
202           id:
203             type: integer
204           title:
205             type: string
206           description:
207             type: ["string", "null"]
208           completed:
209             type: boolean
210           priority:
211             type: string
212             enum: [low, medium, high]
213           dueDate:
214             type: ["string", "null"]
215           createdAt:
216             type: string
217           updatedAt:
218             type: string
219
220       ToDoInput:
221         type: object
222         required:
223           - title
224         properties:
225           title:
226             type: string
227             minLength: 1
228           description:
229             type: string
230           priority:
```

```
231         type: string
232         enum: [low, medium, high]
233     dueDate:
234         type: string
235
236     TodoUpdate:
237         type: object
238         properties:
239             title:
240                 type: string
241                 minLength: 1
242             description:
243                 type: ["string", "null"]
244             completed:
245                 type: boolean
246             priority:
247                 type: string
248                 enum: [low, medium, high]
249             dueDate:
250                 type: ["string", "null"]
251
252     ClearCompletedResult:
253         type: object
254         required:
255             - deleted
256         properties:
257             deleted:
258                 type: integer
259
260     TodoStats:
261         type: object
262         required:
263             - total
264             - completed
265             - active
266             - highPriority
267             - dueSoon
268         properties:
269             total:
270                 type: integer
271             completed:
272                 type: integer
273             active:
274                 type: integer
275             highPriority:
276                 type: integer
277             dueSoon:
278                 type: integer
279
```

API Routes — todos.ts

artifacts/api-server/src/routes/todos.ts

```
1  import { Router, type IRouter } from "express";
2  import { eq } from "drizzle-orm";
3  import { db, todosTable } from "@workspace/db";
4  import {
5    ListTodosQueryParams,
6    ListTodosResponse,
7    CreateTodoBody,
8    GetTodoParams,
9    GetTodoResponse,
10   UpdateTodoParams,
11   UpdateTodoBody,
12   UpdateTodoResponse,
13   DeleteTodoParams,
14   ToggleTodoCompleteParams,
15   ToggleTodoCompleteResponse,
16   GetTodoStatsResponse,
17 } from "@workspace/api-zod";
18
19 const router: IRouter = Router();
20
21 type DbTodo = {
22   id: number;
23   title: string;
24   description: string | null;
25   completed: boolean;
26   priority: string;
27   dueDate: string | null;
28   createdAt: Date;
29   updatedAt: Date;
30 };
31
32 function serializeTodo(todo: DbTodo) {
33   return {
34     ...todo,
35     createdAt: todo.createdAt.toISOString(),
36     updatedAt: todo.updatedAt.toISOString(),
37   };
38 }
39
40 router.delete("/todos/clear-completed", async (_req, res): Promise<void> => {
41   const deleted = await db
42     .delete(todosTable)
43     .where(eq(todosTable.completed, true))
44     .returning();
45
46   res.json({ deleted: deleted.length });
47 });
48
49 router.get("/todos/stats", async (_req, res): Promise<void> => {
50   const now = new Date();
51   const sevenDaysFromNow = new Date(now.getTime() + 7 * 24 * 60 * 60 * 1000);
52   const sevenDaysStr = sevenDaysFromNow.toISOString().split("T")[0];
53
54   const allTodos = await db.select().from(todosTable);
55
56   const total = allTodos.length;
```

```

57     const completed = allTodos.filter((t) => t.completed).length;
58     const active = total - completed;
59     const highPriority = allTodos.filter(
60         (t) => t.priority === "high" && !t.completed
61     ).length;
62     const dueSoon = allTodos.filter(
63         (t) => t.dueDate && t.dueDate <= sevenDaysStr && !t.completed
64     ).length;
65
66     res.json(
67         GetTodoStatsResponse.parse({ total, completed, active, highPriority, dueSoon })
68     );
69 });
70
71 router.get("/todos", async (req, res): Promise<void> => {
72     const parsed = ListTodosQueryParams.safeParse(req.query);
73     if (!parsed.success) {
74         res.status(400).json({ error: parsed.error.message });
75         return;
76     }
77
78     const { status, priority } = parsed.data;
79
80     let todos = await db.select().from(todosTable);
81
82     if (status === "active") {
83         todos = todos.filter((t) => !t.completed);
84     } else if (status === "completed") {
85         todos = todos.filter((t) => t.completed);
86     }
87
88     if (priority) {
89         todos = todos.filter((t) => t.priority === priority);
90     }
91
92     todos.sort(
93         (a, b) => new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime()
94     );
95
96     res.json(ListTodosResponse.parse(todos.map(serializeTodo)));
97 });
98
99 router.post("/todos", async (req, res): Promise<void> => {
100     const parsed = CreateTodoBody.safeParse(req.body);
101     if (!parsed.success) {
102         req.log.warn({ errors: parsed.error.message }, "Invalid request body");
103         res.status(400).json({ error: parsed.error.message });
104         return;
105     }
106
107     const [todo] = await db.insert(todosTable).values(parsed.data).returning();
108     res.status(201).json(GetTodoResponse.parse(serializeTodo(todo)));
109 });
110
111 router.get("/todos/:id", async (req, res): Promise<void> => {
112     const raw = Array.isArray(req.params.id) ? req.params.id[0] : req.params.id;
113     const params = GetTodoParams.safeParse({ id: parseInt(raw, 10) });
114     if (!params.success) {

```

```

115     res.status(400).json({ error: params.error.message });
116     return;
117 }
118
119 const [todo] = await db
120     .select()
121     .from(todosTable)
122     .where(eq(todosTable.id, params.data.id));
123
124 if (!todo) {
125     res.status(404).json({ error: "Todo not found" });
126     return;
127 }
128
129 res.json(GetTodoResponse.parse(serializeTodo(todo)));
130 });
131
132 router.patch("/todos/:id/complete", async (req, res): Promise<void> => {
133     const raw = Array.isArray(req.params.id) ? req.params.id[0] : req.params.id;
134     const params = ToggleTodoCompleteParams.safeParse({ id: parseInt(raw, 10) });
135     if (!params.success) {
136         res.status(400).json({ error: params.error.message });
137         return;
138     }
139
140     const [existing] = await db
141         .select()
142         .from(todosTable)
143         .where(eq(todosTable.id, params.data.id));
144
145     if (!existing) {
146         res.status(404).json({ error: "Todo not found" });
147         return;
148     }
149
150     const [todo] = await db
151         .update(todosTable)
152         .set({ completed: !existing.completed })
153         .where(eq(todosTable.id, params.data.id))
154         .returning();
155
156     res.json(ToggleTodoCompleteResponse.parse(serializeTodo(todo)));
157 });
158
159 router.patch("/todos/:id", async (req, res): Promise<void> => {
160     const raw = Array.isArray(req.params.id) ? req.params.id[0] : req.params.id;
161     const params = UpdateTodoParams.safeParse({ id: parseInt(raw, 10) });
162     if (!params.success) {
163         res.status(400).json({ error: params.error.message });
164         return;
165     }
166
167     const parsed = UpdateTodoBody.safeParse(req.body);
168     if (!parsed.success) {
169         res.status(400).json({ error: parsed.error.message });
170         return;
171     }
172

```

```
173   const [todo] = await db
174     .update(todosTable)
175     .set(parsed.data)
176     .where(eq(todosTable.id, params.data.id))
177     .returning();
178
179   if (!todo) {
180     res.status(404).json({ error: "Todo not found" });
181     return;
182   }
183
184   res.json(UpdateTodoResponse.parse(serializeTodo(todo)));
185 });
186
187 router.delete("/todos/:id", async (req, res): Promise<void> => {
188   const raw = Array.isArray(req.params.id) ? req.params.id[0] : req.params.id;
189   const params = DeleteTodoParams.safeParse({ id: parseInt(raw, 10) });
190   if (!params.success) {
191     res.status(400).json({ error: params.error.message });
192     return;
193   }
194
195   const [todo] = await db
196     .delete(todosTable)
197     .where(eq(todosTable.id, params.data.id))
198     .returning();
199
200   if (!todo) {
201     res.status(404).json({ error: "Todo not found" });
202     return;
203   }
204
205   res.sendStatus(204);
206 });
207
208 export default router;
209
```

App Entry — App.tsx

artifacts/todo-app/src/App.tsx

```
1  import { Switch, Route, Router as WouterRouter } from "wouter";
2  import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
3  import { Toaster } from "@components/ui/toaster";
4  import { TooltipProvider } from "@components/ui/tooltip";
5  import NotFound from "@pages/not-found";
6  import { Layout } from "../components/layout";
7  import { HomePage } from "../pages/home";
8  import { StatsPage } from "../pages/stats";
9  import { CalendarPage } from "../pages/calendar";
10
11  const queryClient = new QueryClient();
12
13  function Router() {
14    return (
15      <Layout>
16        <Switch>
17          <Route path="/" component={HomePage} />
18          <Route path="/stats" component={StatsPage} />
19          <Route path="/calendar" component={CalendarPage} />
20          <Route component={NotFound} />
21        </Switch>
22      </Layout>
23    );
24  }
25
26  function App() {
27    return (
28      <QueryClientProvider client={queryClient}>
29        <TooltipProvider>
30          <WouterRouter base={import.meta.env.BASE_URL.replace(/\$/ /, "")}>
31            <Router />
32          </WouterRouter>
33          <Toaster />
34        </TooltipProvider>
35      </QueryClientProvider>
36    );
37  }
38
39  export default App;
40
```

Layout — layout.tsx

artifacts/todo-app/src/components/layout.tsx

```
1  import { Link, useLocation } from "wouter";
2  import { CheckSquare, BarChart2, Notebook, CalendarDays } from "lucide-react";
3
4  export function Layout({ children }: { children: React.ReactNode }) {
5    const [location] = useLocation();
6
7    return (
8      <div className="min-h-screen flex flex-col md:flex-row bg-background">
9        {/* Sidebar */}
10       <aside className="w-full md:w-64 bg-card border-r border-border shrink-0 flex flex-col">
11         <div className="p-6 md:p-8 flex items-center gap-3">
12           <div className="w-8 h-8 rounded-md bg-primary flex items-center justify-center text-
13             primary-foreground shadow-sm">
14             <Notebook size={18} />
15           </div>
16           <h1 className="text-xl font-serif font-semibold text-foreground">Taskbook</h1>
17         </div>
18
19         <nav className="flex-1 px-4 md:px-6 pb-6 space-y-1">
20           <Link href="/">
21             <div data-testid="link-home" className={`flex items-center gap-3 px-3 py-2.5
22               rounded-md transition-colors cursor-pointer ${location === "/" ? "bg-accent text-accent-
23               foreground font-medium" : "text-muted-foreground hover:text-foreground hover:bg-accent/50"}`}>
24               <CheckSquare size={18} />
25               <span>Tasks</span>
26             </div>
27           </Link>
28
29           <Link href="/calendar">
30             <div data-testid="link-calendar" className={`flex items-center gap-3 px-3 py-2.5
31               rounded-md transition-colors cursor-pointer ${location === "/calendar" ? "bg-accent text-accent-
32               foreground font-medium" : "text-muted-foreground hover:text-foreground hover:bg-accent/50"}`}>
33               <CalendarDays size={18} />
34               <span>Calendar</span>
35             </div>
36           </Link>
37
38           <Link href="/stats">
39             <div data-testid="link-stats" className={`flex items-center gap-3 px-3 py-2.5
40               rounded-md transition-colors cursor-pointer ${location === "/stats" ? "bg-accent text-accent-
41               foreground font-medium" : "text-muted-foreground hover:text-foreground hover:bg-accent/50"}`}>
42               <BarChart2 size={18} />
43               <span>Statistics</span>
44             </div>
45           </Link>
46         </nav>
47       </aside>
48
49       {/* Main Content */}
50       <main className="flex-1 max-w-4xl mx-auto w-full">
51         {children}
52       </main>
53     </div>
54   );
55 }
```

Home Page — home.tsx

artifacts/todo-app/src/pages/home.tsx

```
1  import { useState } from "react";
2  import {
3    useListTodos,
4    getListTodosQueryKey,
5    useGetTodoStats,
6    getGetTodoStatsQueryKey,
7    useClearCompleted,
8  } from "@workspace/api-client-react";
9  import { useQueryClient } from "@tanstack/react-query";
10 import { useToast } from "@hooks/use-toast";
11 import { TodoList } from "../components/todo/todo-list";
12 import { CreateTodo } from "../components/todo/create-todo";
13 import { Button } from "@components/ui/button";
14 import { Tabs, TabsList, TabsTrigger } from "@components/ui/tabs";
15 import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@components/ui/
16 import { ListTodosStatus, ListTodosPriority } from "@workspace/api-client-react";
17
18 export function HomePage() {
19   const [status, setStatus] = useState<ListTodosStatus>("all");
20   const [priority, setPriority] = useState<ListTodosPriority | "all">("all");
21   const queryClient = useQueryClient();
22   const { toast } = useToast();
23
24   const queryParams = {
25     ...(status !== "all" ? { status } : {}),
26     ...(priority !== "all" ? { priority } : {})
27   };
28
29   const { data: todos, isLoading } = useListTodos(queryParams, {
30     query: { queryKey: getListTodosQueryKey(queryParams) }
31   });
32
33   const { data: stats } = useGetTodoStats({
34     query: { queryKey: getGetTodoStatsQueryKey() }
35   });
36
37   const clearCompleted = useClearCompleted({
38     mutation: {
39       onSuccess: (data) => {
40         queryClient.invalidateQueries({ queryKey: getListTodosQueryKey() });
41         queryClient.invalidateQueries({ queryKey: getGetTodoStatsQueryKey() });
42         toast({
43           title: "All clear",
44           description: `${data.deleted} completed task${data.deleted !== 1 ? "s" : ""} removed.
45 See you tomorrow!`,
46         },
47         onError: () => {
48           toast({ title: "Error", description: "Could not clear tasks.", variant:
49 "destructive" });
50         },
51       });
52
53       const allDone = stats && stats.total > 0 && stats.active === 0;
54
55       return (
56         <div className="p-6 md:p-12 space-y-8 animate-in fade-in slide-in-from-bottom-4
```

duration-500">

```

57     <header>
58         <h2 className="text-3xl font-serif font-bold text-foreground">Today's Tasks</h2>
59         <p className="text-muted-foreground mt-2">What needs your attention today?</p>
60     </header>
61
62     {allDone && (
63         <div className="relative overflow-hidden rounded-2xl bg-primary px-8 py-8 text-primary-
64 foreground animate-in fade-in slide-in-from-bottom-4 duration-500">
65         <div className="relative z-10 flex flex-col sm:flex-row sm:items-center justify-
66 between gap-4">
67             <p className="text-xs font-semibold uppercase tracking-widest opacity-75
68 mb-2">All tasks complete</p>
69             <h3 className="text-2xl font-serif font-bold mb-1">Well done!</h3>
70             <p className="text-sm opacity-80">You've finished everything for today. Rest up
71 and come back tomorrow.</p>
72             <Button
73                 variant="outline"
74                 onClick={() => clearCompleted.mutate()}
75                 disabled={clearCompleted.isPending}
76                 className="shrink-0 bg-transparent border-primary-foreground/30 text-primary-
77 foreground hover:bg-primary-foreground/10 hover:text-primary-foreground rounded-xl"
78                 {clearCompleted.isPending ? "Clearing..." : "Clear & start fresh"}
79             </Button>
80         </div>
81         <div className="absolute right-6 top-1/2 -translate-y-1/2 text-[80px] leading-none
82 select-none opacity-10 font-serif pointer-events-none">
83             </div>
84         </div>
85     )}
86
87     <CreateTodo />
88
89     <div className="space-y-6 pt-4 border-t border-border/50">
90         <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
91             <Tabs value={status} onValueChange={(v) => setStatus(v as ListTodosStatus)}>
92                 <TabList className="bg-card border border-border shadow-sm">
93                     <TabTrigger value="all" className="data-[state=active]:bg-primary data-
94 [state=active]:text-primary-foreground">All</TabTrigger>
95                     <TabTrigger value="active" className="data-[state=active]:bg-primary data-
96 [state=active]:text-primary-foreground">Active</TabTrigger>
97                     <TabTrigger value="completed" className="data-[state=active]:bg-primary data-
98 [state=active]:text-primary-foreground">Completed</TabTrigger>
99                 </TabList>
100             </Tabs>
101
102             <Select value={priority} onValueChange={(v) => setPriority(v as ListTodosPriority |
103 "all")}>
104                 <SelectTrigger className="w-[140px] bg-card border border-border shadow-sm">
105                     <SelectValue placeholder="Priority" />
106                 </SelectTrigger>
107                 <SelectContent>
108                     <SelectItem value="all">Any Priority</SelectItem>
109                     <SelectItem value="low">Low Priority</SelectItem>
110                     <SelectItem value="medium">Medium Priority</SelectItem>
111                     <SelectItem value="high">High Priority</SelectItem>
112                 </SelectContent>
113             </Select>
114         </div>
115
116         {isLoading ? (
117             <div className="space-y-4">
118                 {[1, 2, 3].map((i) => (
119                     <div key={i} className="h-20 bg-card rounded-lg border border-border animate-
120 pulse" />
121                 ))}
122             </div>
123         ) : (
124             <div className="space-y-4">
125                 {[1, 2, 3].map((i) => (
126                     <div key={i} className="h-20 bg-card rounded-lg border border-border animate-
127 pulse" />
128                 ))}
129             </div>
130         )}
131     </div>

```

```
115         </div>
116     ) : (
117         <TodoList todos={todos || []} />
118     )}
119 </div>
120 </div>
121 );
122 }
123
```

Calendar Page — calendar.tsx

artifacts/todo-app/src/pages/calendar.tsx

```
1  import { useState } from "react";
2  import {
3    useListTodos,
4    getListTodosQueryKey,
5    getGetTodoStatsQueryKey,
6    useCreateTodo,
7    useToggleTodoComplete,
8    useDeleteTodo,
9  } from "@workspace/api-client-react";
10 import { useQueryClient } from "@tanstack/react-query";
11 import { useToast } from "@hooks/use-toast";
12 import { Button } from "@components/ui/button";
13 import { Input } from "@components/ui/input";
14 import { Badge } from "@components/ui/badge";
15 import {
16   ChevronLeft,
17   ChevronRight,
18   Plus,
19   Check,
20   Trash2,
21   X,
22 } from "lucide-react";
23 import type { Todo } from "@workspace/api-client-react";
24
25 const DAYS = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
26 const MONTHS = [
27   "January", "February", "March", "April", "May", "June",
28   "July", "August", "September", "October", "November", "December",
29 ];
30
31 function toDateStr(d: Date) {
32   return d.toISOString().split("T")[0];
33 }
34
35 function isSameDay(a: Date, b: Date) {
36   return toDateStr(a) === toDateStr(b);
37 }
38
39 const PRIORITY_DOT: Record<string, string> = {
40   high: "bg-rose-500",
41   medium: "bg-amber-400",
42   low: "bg-slate-400",
43 };
44
45 export function CalendarPage() {
46   const today = new Date();
47   const [viewDate, setViewDate] = useState(new Date(today.getFullYear(), today.getMonth(), 1));
48   const [selectedDate, setSelectedDate] = useState<Date>(today);
49   const [newTitle, setNewTitle] = useState("");
50
51   const queryClient = useQueryClient();
52   const { toast } = useToast();
53
54   const { data: todos = [] } = useListTodos(
55     {},
56     { query: { queryKey: getListTodosQueryKey() } }
57   );
```

```

57     );
58
59     const createTodo = useCreateTodo({
60         mutation: {
61             onSuccess: () => {
62                 setNewTitle("");
63                 queryClient.invalidateQueries({ queryKey: getListTodosQueryKey() });
64                 queryClient.invalidateQueries({ queryKey: getGetTodoStatsQueryKey() });
65             },
66             onError: () => {
67                 toast({ title: "Error", description: "Could not add task.", variant: "destructive" });
68             },
69         },
70     });
71
72     const toggleComplete = useToggleTodoComplete({
73         mutation: {
74             onSuccess: () => {
75                 queryClient.invalidateQueries({ queryKey: getListTodosQueryKey() });
76                 queryClient.invalidateQueries({ queryKey: getGetTodoStatsQueryKey() });
77             },
78         },
79     });
80
81     const deleteTodo = useDeleteTodo({
82         mutation: {
83             onSuccess: () => {
84                 queryClient.invalidateQueries({ queryKey: getListTodosQueryKey() });
85                 queryClient.invalidateQueries({ queryKey: getGetTodoStatsQueryKey() });
86             },
87         },
88     });
89
90     const year = viewDate.getFullYear();
91     const month = viewDate.getMonth();
92
93     const firstDay = new Date(year, month, 1).getDay();
94     const daysInMonth = new Date(year, month + 1, 0).getDate();
95
96     const cells: (Date | null)[] = [
97         ...Array(firstDay).fill(null),
98         ...Array.from({ length: daysInMonth }, (_, i) => new Date(year, month, i + 1)),
99     ];
100     while (cells.length % 7 !== 0) cells.push(null);
101
102     function todosForDate(date: Date): Todo[] {
103         const str = toDateStr(date);
104         return todos.filter((t) => t.dueDate === str);
105     }
106
107     const selectedDateStr = toDateStr(selectedDate);
108     const selectedTodos = todos.filter((t) => t.dueDate === selectedDateStr);
109
110     function prevMonth() {
111         setViewDate(new Date(year, month - 1, 1));
112     }
113     function nextMonth() {
114         setViewDate(new Date(year, month + 1, 1));

```

```

115 }
116
117 function handleAddTask(e: React.FormEvent) {
118   e.preventDefault();
119   const trimmed = newTitle.trim();
120   if (!trimmed || createTodo.isPending) return;
121   createTodo.mutate({
122     data: { title: trimmed, priority: "medium", dueDate: selectedDateStr },
123   });
124 }
125
126 function selectDay(date: Date) {
127   setSelectedDate(date);
128 }
129
130 return (
131   <div className="p-6 md:p-10 animate-in fade-in slide-in-from-bottom-4 duration-500">
132     <header className="mb-8">
133       <h2 className="text-3xl font-serif font-bold text-foreground">Calendar</h2>
134       <p className="text-muted-foreground mt-1">Pin major tasks to specific days.</p>
135     </header>
136
137     <div className="grid grid-cols-1 lg:grid-cols-[1fr_320px] gap-6">
138       <div className="bg-card border border-border rounded-2xl shadow-sm overflow-hidden">
139         <div className="flex items-center justify-between px-6 py-4 border-b border-border">
140           <div>
141             <button
142               onClick={prevMonth}
143               className="p-1.5 rounded-md text-muted-foreground hover:text-foreground hover:bg-accent transition-colors"
144             >
145               <ChevronLeft size={18} />
146             </button>
147             <h3 className="text-base font-semibold text-foreground">
148               {MONTHS[month]} {year}
149             </h3>
150             <button
151               onClick={nextMonth}
152               className="p-1.5 rounded-md text-muted-foreground hover:text-foreground hover:bg-accent transition-colors"
153             >
154               <ChevronRight size={18} />
155             </button>
156           </div>
157
158           <div>
159             <div>
160               <div>
161                 <div>
162                   <div>
163                     <div>
164                       <div>
165                         <div>
166                           <div>
167                             <div>
168                               <div>
169                                 <div>
170                                   <div>
171                                     <div>
172                   </div>

```

```

173         {cells.map((date, i) => {
174             if (!date) {
175                 return <div key={`empty-${i}`} className="min-h-[80px] border-b border-r border-
176 border/40 last:border-r-0" />;
177
178                 const dayTodos = todosForDate(date);
179                 const isToday = isSameDay(date, today);
180                 const isSelected = isSameDay(date, selectedDate);
181                 const isCurrentMonth = date.getMonth() === month;
182                 const col = i % 7;
183                 const isLastCol = col === 6;
184
185                 return (
186                     <button
187                         key={toDateStr(date)}
188                         onClick={() => selectDay(date)}
189                         className={`min-h-[80px] p-2 border-b border-r border-border/40 text-left
190 flex flex-col gap-1 transition-colors ${isLastCol ? "border-r-0" : ""}
191 ${isSelected ? "bg-primary/10 ring-1 ring-inset ring-primary/30" :
192 "hover:bg-accent/50"} ${!isCurrentMonth ? "opacity-40" : ""}
193                     `>
194                         >
195                         <span
196                             className={`text-sm font-medium w-7 h-7 flex items-center justify-center
197 rounded-full leading-none ${isToday ? "bg-primary text-primary-foreground" : isSelected ? "text-
198 primary" : "text-foreground"}
199                         >
200                             {date.getDate()}
201                         </span>
202
203                         <div className="flex flex-wrap gap-1 mt-auto">
204                             {dayTodos.slice(0, 3).map((todo) => (
205                                 <span
206                                     key={todo.id}
207                                     className={`w-2 h-2 rounded-full shrink-0 ${PRIORITY_DOT[todo.priority]}
208                                     title={todo.title}
209                                 >/>
210                             ))}
211                             {dayTodos.length > 3 && (
212                                 <span className="text-[10px] text-muted-foreground leading-none self-end">
213                                     +{dayTodos.length - 3}
214                                 </span>
215                             )}
216                         </div>
217
218                         {dayTodos.slice(0, 2).map((todo) => (
219                             <span
220                                 key={todo.id}
221                                 className={`hidden sm:block text-[11px] leading-tight px-1.5 py-0.5
222 rounded-sm truncate max-w-full ${!todo.completed
223                                     ? "line-through text-muted-foreground/60 bg-muted/50"
224                                     : todo.priority === "high"
225                                     ? "bg-rose-50 text-rose-700 dark:bg-rose-950/40 dark:text-rose-300"
226                                     : todo.priority === "medium"
227                                     ? "bg-amber-50 text-amber-700 dark:bg-amber-950/40 dark:text-
228 amber-300"
229                                     : "bg-slate-100 text-slate-600 dark:bg-slate-800 dark:text-slate-300"}
230                             >

```

```

231         {todo.title}
232     </span>
233     )))
234 </button>
235     );
236     )))
237 </div>
238 </div>
239
240     {/ * Day detail panel */}
241     <div className="bg-card border border-border rounded-2xl shadow-sm flex flex-col
242 overflow-hidden" className="px-5 py-4 border-b border-border">
243         <p className="text-xs font-semibold uppercase tracking-widest text-muted-foreground
244 mb-0.5">
245             {isSelectedToday(selectedDate, today) ? "Today" : DAYS[selectedDate.getDay()]}
246         </p>
247         <h4 className="text-xl font-serif font-bold text-foreground">
248             {MONTHS[selectedDate.getMonth()]} {selectedDate.getDate()},
249             {selectedDate.getFullYear()}
250         </h4>
251     </div>
252
253     <div className="flex-1 overflow-y-auto px-4 py-4 space-y-2">
254         {selectedTodos.length === 0 ? (
255             <div className="py-8 text-center text-sm text-muted-foreground">
256                 No tasks pinned to this day.
257             </div>
258         ) : (
259             selectedTodos.map((todo) => (
260                 <div
261                     key={todo.id}
262                     className={`flex items-start gap-2 p-3 rounded-lg border transition-colors
263                         ${todo.completed ? "bg-muted/30 border-border/50 opacity-60" : "bg-
264 background border-border border hover:border-primary/30"}
265                 >
266                     <button
267                         onClick={() => toggleComplete.mutate({ id: todo.id })}
268                         className={`shrink-0 mt-0.5 w-5 h-5 rounded border flex items-center
269 justify-center transition-colors ${todo.completed ? "bg-primary border-primary text-primary-foreground" :
270 "border-input hover:border-primary/50"}
271                     >
272                         {todo.completed && <Check size={12} />}
273                     </button>
274                     <div className="flex-1 min-w-0">
275                         <p className={`text-sm font-medium leading-snug ${todo.completed ? "line-
276 through text-muted-foreground" : "text-foreground"} `}>
277                             {todo.title}
278                         </p>
279                         <Badge
280                             variant="outline"
281                             className={`mt-1 text-[10px] font-normal px-1.5 py-0
282                                 ${todo.priority === "high" ? "text-rose-600 border-rose-200 bg-rose-50
283 dark:bg-rose-950/40 dark:text-rose-300 dark:border-rose-900" : "text-amber-600 border-amber-200 bg-
284 amber-50 dark:bg-amber-950/40 dark:text-amber-300 dark:border-amber-900"
285                                 dark:text-slate-300 dark:border-slate-700`} `}
286                             {todo.priority}
287                         </Badge>
288                     </div>
289                     <button
290                         onClick={() => deleteTodo.mutate({ id: todo.id })}
291                         className="shrink-0 p-1 text-muted-foreground hover:text-destructive
292 rounded transition-colors"

```

```

289         >
290         <Trash2 size={13} />
291     </button>
292 </div>
293 ))
294 }}
295 </div>
296
297 {/* Add task form */}
298 <div className="px-4 pb-4 pt-2 border-t border-border mt-auto">
299     <form onSubmit={handleAddTask} className="flex gap-2">
300         <Input
301             type="text"
302             placeholder="Add task for this day..."
303             value={newTitle}
304             onChange={(e) => setNewTitle(e.target.value)}
305             disabled={createTodo.isPending}
306             className="flex-1 text-sm bg-background"
307         />
308         <Button
309             type="submit"
310             size="icon"
311             disabled={!newTitle.trim() || createTodo.isPending}
312             className="shrink-0 rounded-lg"
313         >
314             <Plus size={16} />
315         </Button>
316     </form>
317 </div>
318 </div>
319 </div>
320 </div>
321 );
322 }
323
324 function isSelectedToday(selected: Date, today: Date) {
325     return (
326         selected.getFullYear() === today.getFullYear() &&
327         selected.getMonth() === today.getMonth() &&
328         selected.getDate() === today.getDate()
329     );
330 }
331

```

Stats Page — stats.tsx

artifacts/todo-app/src/pages/stats.tsx

```
1  import { useGetTodoStats, getGetTodoStatsQueryKey } from "@workspace/api-client-react";
2  import { CheckCircle2, Clock, AlertCircle, LayoutList } from "lucide-react";
3
4  export function StatsPage() {
5    const { data: stats, isLoading } = useGetTodoStats({
6      query: { queryKey: getGetTodoStatsQueryKey() }
7    });
8
9    if (isLoading || !stats) {
10     return (
11       <div className="p-6 md:p-12 space-y-8 animate-in fade-in duration-500">
12         <div className="h-10 w-48 bg-muted rounded animate-pulse" />
13         <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
14           {[1, 2, 3, 4].map((i) => (
15             <div key={i} className="h-32 bg-card rounded-xl border border-border animate-
16 pulse" /> ))}
17         </div>
18       </div>
19     );
20   }
21
22   const completionRate = stats.total > 0 ? Math.round((stats.completed / stats.total) * 100) :
23   0;
24   return (
25     <div className="p-6 md:p-12 space-y-8 animate-in fade-in slide-in-from-bottom-4
26 duration-500">
27       <header>
28         <h2 className="text-3xl font-serif font-bold text-foreground">Productivity Summary</h2>
29         <p className="text-muted-foreground mt-2">A high-level view of your focus and
30 accomplishments.</p>
31       </header>
32       <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
33         <StatCard
34           title="Total Tasks"
35           value={stats.total}
36           icon={<LayoutList size={24} className="text-primary" />}
37           description="Tasks created overall"
38           delay={0}
39         />
40         <StatCard
41           title="Completed"
42           value={stats.completed}
43           icon={<CheckCircle2 size={24} className="text-emerald-500" />}
44           description={` ${completionRate}% completion rate`}
45           delay={100}
46         />
47         <StatCard
48           title="Active Focus"
49           value={stats.active}
50           icon={<Clock size={24} className="text-amber-500" />}
51           description="Tasks currently in progress"
52           delay={200}
53         />
54         <StatCard
55           title="High Priority"
56           value={stats.highPriority}
57           icon={<AlertCircle size={24} className="text-rose-500" />}
58         />
59       </div>
60     </div>
61   );
62 }
```

```

57         description="Items needing immediate attention"
58         delay={300}
59     />
60 </div>
61
62     {stats.dueSoon > 0 && (
63         <div className="mt-8 p-6 bg-amber-50 dark:bg-amber-950/20 border border-amber-200
64         dark:border-amber-900 rounded-xl flex items-start gap:4 animate-in fade-in zoom-in duration-500
65         delay-500 fill-mode-both">
66             <div>
67                 <h3 className="font-semibold text-amber-900 dark:text-amber-400">Deadlines
68                 approaching</h3>
69                 <p3 className="text-amber-800/80 dark:text-amber-500/80 mt-1">
69                     You have {stats.dueSoon} task{stats.dueSoon === 1 ? '' : 's'} due soon. Check
69                     your active tasks to ensure nothing slips through the cracks.
70                 </p>
71             </div>
72         </div>
73     )}
74 </div>
75 );
76 }
77
78 function StatCard({ title, value, icon, description, delay }: { title: string, value: number,
78 icon: React.ReactNode, description: string, delay: number }) {
79     <div
80         className="bg-card border border-border p-6 rounded-xl shadow-sm hover:shadow-md
81         transition-shadow animate-in fade-in slide-in from-bottom-4 duration-500 fill-mode-both"
82         style={{ animationDelay: `${delay}ms` }}
83     >
84         <div className="flex items-start justify-between">
85             <div>
86                 <p className="text-sm font-medium text-muted-foreground">{title}</p>
87                 <p className="text-4xl font-serif font-semibold text-foreground mt-2">{value}</p>
88             </div>
89             <div className="p-3 bg-muted/50 rounded-lg">
90                 {icon}
91             </div>
92         </div>
93         <p className="text-sm text-muted-foreground mt-4">{description}</p>
94     </div>
95 );
96 }

```

Create Todo — create-todo.tsx

artifacts/todo-app/src/components/todo/create-todo.tsx

```
1  import { useState } from "react";
2  import { useCreateTodo, getListTodosQueryKey, getGetTodoStatsQueryKey } from "@workspace/api-client";
3  import { useQueryClient } from "@tanstack/react-query";
4  import { useToast } from "@hooks/use-toast";
5  import { Button } from "@components/ui/button";
6  import { Input } from "@components/ui/input";
7  import {
8    Select,
9    SelectContent,
10   SelectItem,
11   SelectTrigger,
12   SelectValue,
13 } from "@components/ui/select";
14
15 type Priority = "low" | "medium" | "high";
16
17 export function CreateTodo() {
18   const [title, setTitle] = useState("");
19   const [priority, setPriority] = useState<Priority>("medium");
20   const queryClient = useQueryClient();
21   const { toast } = useToast();
22
23   const createTodo = useCreateTodo({
24     mutation: {
25       onSuccess: () => {
26         setTitle("");
27         queryClient.invalidateQueries({ queryKey: getListTodosQueryKey() });
28         queryClient.invalidateQueries({ queryKey: getGetTodoStatsQueryKey() });
29         toast({ title: "Task added", description: "Your task has been created." });
30       },
31       onError: () => {
32         toast({ title: "Error", description: "Failed to add task. Please try again.", variant:
33 "destructive" });
34       },
35     });
36
37   const handleSubmit = (e: React.FormEvent) => {
38     e.preventDefault();
39     const trimmed = title.trim();
40     if (!trimmed || createTodo.isPending) return;
41     createTodo.mutate({ data: { title: trimmed, priority } });
42   };
43
44   return (
45     <form onSubmit={handleSubmit}>
46       <div className="flex flex-col sm:flex-row gap-2">
47         <Input
48           type="text"
49           placeholder="Add a new task..."
50           value={title}
51           onChange={(e) => setTitle(e.target.value)}
52           disabled={createTodo.isPending}
53           className="flex-1 py-5 text-base bg-card border-border shadow-sm focus-visible:ring-
54 primary/20 rounded-xl"
55         />
56         <Select value={priority} onChange={(v) => setPriority(v as Priority)}>
57           <SelectTrigger className="w-full sm:w-36 bg-card border-border shadow-sm rounded-xl">
```

```
57         <SelectValue />
58     </SelectTrigger>
59     <SelectContent>
60         <SelectItem value="low">Low</SelectItem>
61         <SelectItem value="medium">Medium</SelectItem>
62         <SelectItem value="high">High</SelectItem>
63     </SelectContent>
64 </Select>
65 <Button
66     type="submit"
67     disabled={!title.trim() || createTodo.isPending}
68     className="rounded-xl font-medium px-6"
69 >
70     {createTodo.isPending ? "Adding..." : "Add Task"}
71 </Button>
72 </div>
73 </form>
74 );
75 }
76
```

Todo Item — todo-item.tsx

artifacts/todo-app/src/components/todo/todo-item.tsx

```
1  import { useState } from "react";
2  import { Todo } from "@workspace/api-client-react";
3  import { useToggleTodoComplete, useDeleteTodo, useUpdateTodo, getListTodosQueryKey,
4  useGetTodoStatsQueryKey } from "@workspace/api-client-react";
5  import { Check, Trash2, Calendar, Edit2, X } from "lucide-react";
6  import { format } from "date-fns";
7  import { Badge } from "@components/ui/badge";
8
9  export function TodoItem({ todo, index }: { todo: Todo, index: number }) {
10   const queryClient = useQueryClient();
11   const [isEditing, setIsEditing] = useState(false);
12   const [editTitle, setEditTitle] = useState(todo.title);
13
14   const toggleComplete = useToggleTodoComplete({
15     mutation: {
16       onSuccess: () => {
17         queryClient.invalidateQueries({ queryKey: getListTodosQueryKey() });
18         queryClient.invalidateQueries({ queryKey: useGetTodoStatsQueryKey() });
19       }
20     }
21   });
22
23   const deleteTodo = useDeleteTodo({
24     mutation: {
25       onSuccess: () => {
26         queryClient.invalidateQueries({ queryKey: getListTodosQueryKey() });
27         queryClient.invalidateQueries({ queryKey: useGetTodoStatsQueryKey() });
28       }
29     }
30   });
31
32   const updateTodo = useUpdateTodo({
33     mutation: {
34       onSuccess: () => {
35         queryClient.invalidateQueries({ queryKey: getListTodosQueryKey() });
36         setIsEditing(false);
37       }
38     }
39   });
40
41   const handleToggle = () => {
42     toggleComplete.mutate({ id: todo.id });
43   };
44
45   const handleDelete = () => {
46     deleteTodo.mutate({ id: todo.id });
47   };
48
49   const handleSave = () => {
50     if (editTitle.trim() && editTitle !== todo.title) {
51       updateTodo.mutate({ id: todo.id, data: { title: editTitle } });
52     } else {
53       setIsEditing(false);
54     }
55   };
56 }
```

```

57   const priorityColors = {
58     low: "bg-slate-100 text-slate-700 border-slate-200 dark:bg-slate-900 dark:text-slate-400
59     dark:border-slate-800" medium: "bg-amber-50 text-amber-700 border-amber-200 dark:bg-amber-950/50 dark:text-
60     amber-400 dark:border-amber-900" high: "bg-rose-50 text-rose-700 border-rose-200 dark:bg-rose-950/50 dark:text-rose-400
61     dark:border-rose-900"
62   },
63   return (
64     <div
65       className={`group relative flex items-start gap-4 p-4 md:p-5 bg-card border border-border
66       rounded-xl shadow-sm transition-all duration-300 animate-in fade-in slide-in-from-bottom-2 fill-
67       mode-both ${todo.completed ? 'opacity-60 bg-muted/30' : 'hover:shadow-md hover:border-primary/30'}
68       style={{ animationDelay: `${index * 50}ms` }}
69     >
70       <button
71         onClick={handleToggle}
72         disabled={toggleComplete.isPending}
73         className={`shrink-0 w-6 h-6 mt-0.5 rounded flex items-center justify-center border
74         transition-colors ${todo.completed
75           ? 'bg-primary border-primary text-primary-foreground'
76           : 'border-input hover:border-primary/50 text-transparent'}
77       `}
78     >
79       <Check size={14} className={todo.completed ? "animate-in zoom-in duration-200" : ""} />
80     </button>
81
82     <div className="flex-1 min-w-0">
83       {isEditing ? (
84         <div className="flex items-center gap-2">
85           <input
86             autoFocus
87             type="text"
88             value={editTitle}
89             onChange={(e) => setEditTitle(e.target.value)}
90             onKeyDown={(e) => e.key === 'Enter' && handleSave()}
91             className="flex-1 bg-background border border-input rounded-md px-3 py-1 text-sm
92             focus:outline-none focus:ring-2 focus:ring-primary/20"
93           <button onClick={handleSave} className="p-1.5 text-primary hover:bg-primary/10
94             rounded-md"> <Check size={16} />
95           </button>
96           <button onClick={() => { setIsEditing(false); setEditTitle(todo.title); }}
97             className="p-1.5 text-muted-foreground hover:bg-muted rounded-md">
98             <X size={16} />
99           </button>
100         </div>
101       ) : (
102         <h3 className={`text-base font-medium transition-colors ${todo.completed ? 'line-
103         through text-muted-foreground' : 'text-foreground'} `}>
104           {todo.title}
105         </h3>
106       )}
107
108       {todo.description && !isEditing && (
109         <p className={`text-sm mt-1 line-clamp-2 ${todo.completed ? 'text-muted-
110         foreground/70' : 'text-muted-foreground'} `}>
111           {todo.description}
112         </p>
113       )}
114
115       <div className="flex flex-wrap items-center gap-3 mt-3">
116         <Badge variant="outline" className={`text-xs font-normal
117         ${priorityColors[todo.priority]} `}>
118           {priorityColors[todo.priority].charAt(0).toUpperCase() + todo.priority.slice(1)} Priority

```

```

115         </Badge>
116
117         {todo.dueDate && (
118             <div className="flex items-center gap-1.5 text-xs text-muted-foreground">
119                 <Calendar size={12} />
120                 <span>{format(new Date(todo.dueDate), 'MMM d, yyyy')}</span>
121             </div>
122         )}
123     </div>
124 </div>
125
126     <div className="opacity-0 group-hover:opacity-100 transition-opacity flex items-center
127 gap-1"> {!todo.completed && !isEditing && (
128         <button
129             onClick={() => setIsEditing(true)}
130             className="p-2 text-muted-foreground hover:text-primary hover:bg-primary/10 rounded-
131 md transition-colors"
132             <Edit2 size={16} />
133         </button>
134     )}
135     <button
136         onClick={handleDelete}
137         disabled={deleteTodo.isPending}
138         className="p-2 text-muted-foreground hover:text-destructive hover:bg-destructive/10
139 rounded-md transition-colors"
140         <Trash2 size={16} />
141     </button>
142 </div>
143 </div>
144 );
145 }
146

```

Todo List — todo-list.tsx

artifacts/todo-app/src/components/todo/todo-list.tsx

```
1  import { Todo } from "@workspace/api-client-react";
2  import { TodoItem } from "../todo-item";
3  import { FileCheck } from "lucide-react";
4
5  export function TodoList({ todos }: { todos: Todo[] }) {
6    if (todos.length === 0) {
7      return (
8        <div className="py-16 text-center border border-dashed border-border rounded-xl bg-
9card/50"><div className="w-16 h-16 mx-auto bg-muted rounded-full flex items-center justify-
10center text-muted-foreground mb-4">
11      <FileCheck size={32} />
12    </div>
13    <h3 className="text-lg font-serif font-medium text-foreground">All caught up</h3>
14    <p className="text-sm text-muted-foreground mt-2 max-w-[250px] mx-auto">
15      You don't have any tasks here. Take a break or add a new task above.
16    </p>
17  </div>
18  );
19  }
20
21  return (
22    <div className="space-y-3">
23      {todos.map((todo, index) => (
24        <TodoItem key={todo.id} todo={todo} index={index} />
25      ))}
26    </div>
27  );
28 }
```